Microservices Interview Questions

Topic-wise Organized Reference Guide

Topics: Basics & Architecture • Communication • Design Patterns • Spring Cloud • Security • Fault Tolerance • Advanced Concepts

1. Basics & Architecture

Q: What are the types of applications?

There are two main types of application architectures:

- Monolith Application
- Microservices Application

Q: What is a Monolith application?

A monolith architecture means the application is built as a single unit. Such an application comprises a client-side interface, a server-side application, and a single database.

Q: What are the disadvantages of a Monolith?

- **Codebase Size:** The codebase gets larger in size with time, making it very difficult to manage.
- **Technology Lock-in:** It is very difficult to introduce new technology as it affects the whole application.
- **Scaling Issues:** It is very difficult to scale a single module; one has to scale the whole application.

Q: What are Microservices?

Microservices is an architectural style where an application is divided into small, independent services, each handling a specific business function. These services communicate over APIs and can be developed, deployed, and scaled independently.

Q: What are Microservices? (Interview Definition)

Microservices are an architecture where the application is loosely coupled, and services can be independently developed, deployed, and maintained. Each service performs a unique function.

Q: What is the specialty of Microservice architecture?

The specialty is the **Polyglot architecture** — it supports microservices with different languages. Example: One team works on a microservice with Java, Spring Boot, and PostgreSQL. Another can work on a microservice using Python, Node.js, and NoSQL.

Q: Why Microservices?

To solve the problems found in Monolith applications, such as scaling difficulties and slow deployment cycles.

Q: When to use Microservices?

- To reduce Time to Market.
- To create Stable/Better software.
- To achieve Lower costs and Faster development.
- For Cloud-native development.
- Complex Applications: Your app is large with many distinct functions.
- Scalability Needs: Different parts of the app need to scale independently.
- Multiple Teams: Several teams work on different parts of the app.
- Fault Tolerance: You need to isolate failures to avoid impacting the entire system.

Q: When to avoid Microservices?

- **Simple Applications:** Small or less complex apps might not need the overhead.
- **Small Teams:** Limited resources might struggle with the complexity of managing microservices.

Q: What is the difference between Monolithic, SOA, and Microservices?

Monolithic: Single unit, tightly coupled — all software components assembled together in a big container.
SOA: A collection of services which communicate with each other. Can involve simple data passing or coordinating some activity. Shares and reuses as much as possible.
Microservices: Highly decoupled, independent databases, fine-grained services. Focuses on sharing as little as possible.
Main diff between SOA and Microservices is based on sharing of data — SOA shares and reuses as much as possible while Microservices focuses on sharing as little as possible.

Q: What are the Pros of Microservice architecture?

- Freedom to use different technologies.
- Each microservice focuses on a single capability.
- Supports individual deployable units.
- Allows frequent software releases.
- Fault Isolation: Issues in one service don't impact the whole system.
- Improved Maintainability: Smaller codebases are easier to manage.

Q: What are the Cons of Microservice architecture?

- Management of a large number of services is difficult.
- Communication between microservices is complex.
- Increased efforts for configuration and other operations.
- Difficult to maintain transaction safety and data boundaries.
- Data consistency issues (Distributed Transactions).
- Network latency and complex debugging.

Q: What are the main features of Microservices?

- Decentralization
- Security
- Polyglot

Q: What are the Main Components of Java Spring Boot Microservices?

- **Services:** The heart of microservices where your business logic lives.
- **API Gateway:** Provides a way for services to communicate with each other.
- **Database:** Where your application data is stored.
- **Monitoring:** Ensures microservices are running properly and collects data for analysis.
- **Schedulers:** Control when services run and how they interact.
- Cloud Infrastructure, Containerization and Orchestration, Message Broker, Security, Service Registry.

2. Communication

Q: How do Microservices communicate with each other?

There are 2 different types of inter-service communication:

1. **Synchronous Communication**
2. **Asynchronous Communication**

Q: What are the ways to communicate in Microservices? (Technical)

- **Synchronous:** REST API, gRPC, GraphQL.
- **Asynchronous:** Message Queues (Kafka, RabbitMQ, Amazon SQS), Event Streams.
- **RPC Framework:** gRPC uses RPC frameworks for communication purposes.

Q: What is Synchronous communication?

The client sends a request and waits for a response from the service. The thread is blocked until the response is received. (REST/gRPC — high coupling.)

Q: What is Asynchronous communication?

It refers to the exchange of data between services without waiting for an immediate response. Services send messages or events via message queues/event streams and continue their work. Better for decoupling and handling spikes. (Kafka/RabbitMQ.)

Q: What is Event-Driven Architecture?

A design where services communicate by emitting and consuming events. For example, an Order service emits an OrderCreated event, which the Inventory and Shipping services pick up.

Q: What is gRPC vs REST?

Use gRPC for high-performance internal service-to-service communication. REST is used for lightweight HTTP communication between two services.

Q: What is Backpressure?

What happens if your Producer (Kafka) is sending messages faster than your Consumer can process them. Backpressure is a mechanism to handle this overload situation.

3. Design Patterns

Q: What are the various design patterns used in Microservices?

- Aggregator
- API Gateway
- Service Discovery
- Asynchronous Messaging
- Event Sourcing
- Saga
- Circuit Breaker
- CQRS
- Database per Service
- Bulkhead

Q: What is the SAGA pattern?

A microservice design pattern used to split big tasks into small local transactions.
Each saga has 2 jobs to do:
1. Update the current microservice and make required changes.
2. Publish events to trigger the next transaction for the next microservice.

Example: In a system like Zomato — Order → Payment → Delivery. Each one has its own database or different transactions.

Q: How does Saga handle failure of an individual saga?

The Saga pattern provides transaction management using a sequence of local transactions. Every microservice has its own database and is able to manage local transactions atomically. If one fails, **compensating transactions** are used to maintain eventual consistency.

Q: How many ways can Saga implementation be done?

There are 2 ways:
1. **Choreography**
2. **Orchestration**

Q: What is the Choreography Saga pattern?

Choreography is a way to coordinate sagas where participants exchange events without a centralized point of control. Each microservice runs its own local transaction and publishes events to a Message Broker.

Advantages:

- Good for simple workflows that require few participants.
- They don't need coordination logic.
- Do not require any additional service implementation and maintenance.

Q: What is the Orchestration Saga pattern?

A way to coordinate sagas where a centralized controller (the Orchestrator) tells the saga participants what local transactions to execute. The Saga Orchestrator handles all transactions, stores them, and interprets the state of each task.

Q: What is Circuit Breaker pattern?

A circuit breaker is a design pattern used to handle failures gracefully and prevent cascading failures across dependent services. It acts as a switch that opens (breaks the circuit) when a service call fails repeatedly, stopping further requests to the failing service until it recovers.

Key States:

1. **Closed State:** All requests go through normally.
2. **Open State:** Requests are blocked and a fallback response is returned.
3. **Half-Open State:** Limited requests are allowed to check if the service has recovered.

Tool: Resilience4j.

Q: What is the Bulkhead pattern?

Isolating resources (like thread pools) so that if one service fails, it doesn't consume all the memory/threads of the entire system.

Q: What is CQRS?

Command Query Responsibility Segregation — separates the Read and Write operations. It separates your Write database from your Read database to improve performance and optimizes each operation separately for efficiency.

Q: What is Database-per-service?

Each service must have its own private database to ensure loose coupling. No other service can access this database directly. This ensures data isolation and enables scaling and individual development.

Q: What is the Strangler Fig Pattern?

Used when migrating a monolith to Microservices. You don't rewrite it all at once. You wrap the monolith and slowly 'strangle' it by replacing one feature at a time with a microservice until the monolith is gone.

Q: What is Blue-Green Deployment?

Running two identical production environments and switching traffic between them to deploy new versions with zero downtime.

Q: What is Canary Deployment?

Rolling out a new version to only 5% of users first to check for bugs before a full rollout.

4. Spring Cloud & Tools

Q: What is Spring Cloud, and how does it help in Microservices?

Spring Cloud is a collection of tools for building and managing microservices. It simplifies common tasks like:

1. **Service Discovery:** Enables services to find each other (e.g., using Eureka).
2. **Load Balancing:** Distributes traffic across service instances (e.g., with Ribbon/Spring Cloud LoadBalancer).
3. **Fault Tolerance:** Protects services from failures (e.g., with Hystrix/Resilience4j).
4. **Centralized Configuration:** Manages configurations across services (e.g., using Spring Cloud Config).
5. **API Gateway:** Provides a single entry point for routing requests (e.g., using Spring Cloud Gateway).

Q: Why do we really need Service Discovery?

One microservice can discover the URL address of another microservice. Each microservice registers its address with the discovery server.

It involves:

1. **Service Registration:** Microservices register themselves with a central registry (e.g., Eureka, Consul) when they start, providing their location (address and port).
2. **Service Lookup:** When a service needs to communicate with another, it queries the registry to find the current location of the target service.
3. **Dynamic Updates:** The registry is updated dynamically as services are added or removed.

Q: What is Netflix Eureka?

A Service Registry. Instances register with Eureka, and clients query Eureka to get the physical location (IP/Port) of the service they want to call.

Q: What is API Gateway?

The API Gateway sits between the client and microservices. It acts as the entry point for all microservices. All HTTP calls should go through the API Gateway.

- **Load Balancing:** Distributes incoming requests to ensure even load.
- **Authentication and Authorization:** Manages access control and security.
- **Rate Limiting:** Limits requests to prevent abuse.
- **Framework for implementation:** Zuul (OLD), Spring Cloud Gateway.

Q: What is Feign Client?

A declarative web service client. Instead of writing boilerplate RestTemplate code, you define an interface with annotations, and Spring handles the implementation.

Q: What is Zuul vs Spring Cloud Gateway?

Zuul (1.0) was blocking/synchronous. Spring Cloud Gateway is the modern replacement built on Spring WebFlux, making it non-blocking and more efficient.

Q: What is Spring Cloud Config Server?

Provides server and client-side support for externalized configuration in a distributed system. You can read the properties from a Git project. It pulls configuration from a Git repository or File System and provides it to all microservices at startup or via a refresh bus.

Q: What is Configuration Management?

The practice of managing environment-specific properties (DB URLs, Secret Keys) outside the code using a central tool like Spring Cloud Config Server.

Q: What is Centralized Logging?

Collecting logs from all distributed services into one searchable dashboard (e.g., ELK Stack — Elasticsearch, Logstash, Kibana — or Splunk) to debug issues that span across multiple services.

Q: What is Distributed Tracing?

Tracking a single request as it travels through multiple services using a Trace ID to link all logs across services. Tools: Spring Cloud Sleuth/Micrometer and Zipkin/Jaeger.

5. Security

Q: How do you implement Authentication and Authorization in Microservices?

Using OAuth 2.0 and JWT:

1. **Authentication:** Use an Identity Provider (like Keycloak or Auth0) to authenticate users. The provider issues a JWT token upon login.
2. **Authorization:** Microservices validate the JWT to check the user's roles and permissions, ensuring they can access the requested resource.
3. **API Gateway:** The API Gateway validates the JWT before routing requests to the microservices.
4. **Spring Security:** For token validation and enforcing access control policies like role-based authorization.

Q: What is JWT?

JSON Web Token — used to pass user identity between services without re-authenticating at every step. A user logs in, gets a JWT, and the API Gateway or individual services validate this token without needing to call the Auth server every time.

Q: What is mTLS?

Mutual TLS — how services prove their identity to each other so a hacker can't spoof a service inside your network.

Q: What is API Versioning Strategy?

Handling changes without breaking clients. Common methods:

- URI versioning (/v1/users)
- Header versioning
- Query Params versioning

6. Fault Tolerance & Resilience

Q: How do you handle Fault Tolerance in Microservices?

- Fault tolerance is used to avoid cascading failures.
- A microservice can continue if another microservice fails in the system.
- In case of failure, return a default response from a fallback method.

Q: What is the Retry Mechanism?

Automatically re-attempting a failed request (usually for transient network issues). It should always be used with exponential backoff to avoid spamming a struggling service.

```
@Retryable(value = {SomeTemporaryException.class}, maxAttempts = 5, backoff =
@Backoff(delay = 2000))
```

Q: What is Rate Limiting?

Limiting the number of requests a user can make in a timeframe to prevent abuse.

Q: What is the Service Mesh?

Tools like Istio or Linkerd that handle service communication, security, and retries automatically.

Q: What type of challenges would you face in Microservices?

- **Complexity:** Managing many services.
- **Data Consistency:** Keeping data synced across services.
- **Communication:** Handling inter-service calls.
- **Deployment:** Continuous deployment and monitoring.
- **Security:** Ensuring secure communication.

7. Data Consistency & Transactions

Q: What is a Transaction in Microservices?

A series of operations that must either all succeed or all fail to keep data consistent. Because microservices are distributed, handling transactions across services is more complex than in monolithic apps.

Local Transaction: Each microservice handles its own database transaction locally using @Transactional.

Distributed Transaction: When multiple microservices are involved, we use the Saga Pattern (Choreography or Orchestration) since distributed transactions like 2PC are not recommended. In that case, use Kafka event listeners and senders.

Q: How do you manage Data Consistency in Microservices?

- **Eventual Consistency with Sagas:** Use Saga Pattern (Choreography or Orchestration) to maintain consistency across distributed services.
- **Database Per Microservice:** Each service should have its own database to avoid tight coupling.
- Use CQRS for read/write separation.

Q: What is the CAP Theorem?

The trade-off between Consistency, Availability, and Partition Tolerance. In Microservices, we usually pick AP (Availability and Partition Tolerance) and settle for Eventual Consistency.

Q: What is Idempotency?

The ability of an API to be called multiple times with the same input without changing the result beyond the initial call.
Example: If a 'Payment Service' receives the same 'Pay $50' message twice due to a network glitch, idempotency ensures the user isn't charged twice. Hint: Use unique Transaction IDs.

Q: What is CQRS (Command Query Responsibility Segregation)?

Separating the Read and Write operations. It optimizes each operation separately for efficiency — separate Write database from Read database to improve performance.

Q: What is Database Sharding?

Splitting a massive database into smaller chunks based on a key (like UserID) to improve performance and scalability.

8. Advanced Concepts

Q: What is the end-to-end scalable design flow?

Client → DNS → Load Balancer → API Gateway → Service Discovery → Microservice (with Circuit Breaker/Bulkhead) → Distributed Cache (Redis) → Database (Sharded/Replicated) → Async Messaging (Kafka) for downstream tasks.

Q: What is Containerization (Docker/Kubernetes)?

Docker: Packages the code and dependencies into an image for consistency.
Kubernetes (K8s): Manages those containers — handling scaling, health checks, and automated rollouts.

Q: What is Load Balancing?

Distributing incoming network traffic across multiple instances of a service to ensure no single instance is overwhelmed. In Spring, we use Spring Cloud LoadBalancer.

Q: Explain the design patterns of Java Spring Boot Microservices.

- **Service Registry and Discovery:** Services automatically register in a central registry, allowing others to identify and interact with them dynamically.
- **API Gateway:** Acts as a customer entry point and forwards requests to appropriate microservices. Provides authentication and rate limits.
- **Circuit Breaker:** Monitors service availability and protects from failures by sending requests or providing fallback responses.
- **CQRS:** Separates read and write operations and optimizes each operation separately for efficiency.
- **Saga Pattern:** Manages distributed tasks by organizing a sequence of local transactions.
- **Database per Service:** Each service has its own separate database ensuring data isolation.
- **Asynchronous Messaging:** Services communicate through message queues like Kafka or RabbitMQ.

Q: What are Metrics and Monitoring?

When you have 50 services, logging isn't enough:

1. **Distributed Tracing:** Using a Trace ID to follow a single user request as it travels through multiple services (Zipkin, Jaeger).
2. **Metrics:** Monitoring CPU, memory, and request latency (Prometheus, Grafana).
3. **Centralized Logging:** Aggregating all logs into one searchable place (ELK Stack).

Q: How to start with Microservices from a Monolith?

If you have a monolith application, you must identify all possible standalone functionalities. Once identified, you need to create standalone projects (often using Spring Boot) to create these microservices. Use the Strangler Fig Pattern to wrap the monolith and slowly replace one feature at a time.